Short Course in MATLAB

Lecture 3

Amit Harlev

Quick Practice

Write a function that counts the number of odd numbers in an array of integers using a for loop and `mod(a,m)`.

After you write the function, manually test it out on a few different arrays.

Quick Practice

Write a function that counts the number of odd numbers in an array of integers using a for loop and `mod(a,m)`.

After you write the function, manually test it out on a few different arrays.

```
function numOdd = countOddNums(numbers)
% Counts the number of odd integers in integer array numbers

  numOdd = 0;
  for num = numbers
    numOdd = numOdd + (mod(num, 2) == 1);
  end
end
```

2-D arrays: Matrices

- A 2-D array is a table called a **matrix**.

2-D arrays: Matrices

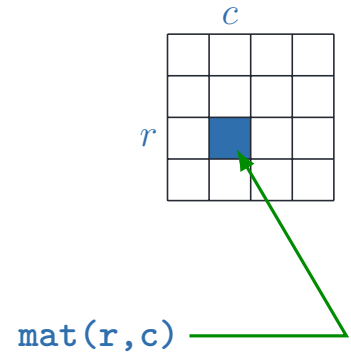
- A 2-D array is a table called a **matrix**.
- Two **indices** identify the position of each value: one row and one column.

2-D arrays: Matrices

- A 2-D array is a table called a **matrix**.
- Two **indices** identify the position of each value: one row and one column.
- Matrices are **rectangular**: every row has the same number of columns.

2-D arrays: Matrices

- A 2-D array is a table called a **matrix**.
- Two **indices** identify the position of each value: one row and one column.
- Matrices are **rectangular**: every row has the same number of columns.
- Reminder: MATLAB array indices start at **1**.



refers to the value in row `r`, column `c` of matrix `mat`.

Creating a Matrix

- Built in functions: `zeros`, `ones`, `rand`
  - e.g. `zeros(2,3)` creates a matrix of all zeros with 2 rows and 3 columns:

$$\begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix}$$

Creating a Matrix

- Built in functions: `zeros`, `ones`, `rand`
  - e.g. `zeros(2,3)` creates a matrix of all zeros with 2 rows and 3 columns:

$$\begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix}$$

- `[1 2 3]` creates the row vector: $\begin{bmatrix} 1 & 2 & 3 \end{bmatrix}$

Creating a Matrix

- Built in functions: `zeros`, `ones`, `rand`
  - e.g. `zeros(2,3)` creates a matrix of all zeros with 2 rows and 3 columns:

$$\begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix}$$

- `[1 2 3]` creates the row vector: $\begin{bmatrix} 1 & 2 & 3 \end{bmatrix}$
- `[1; 2; 3]` creates the column vector: $\begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix}$

Creating a Matrix

- Built in functions: `zeros`, `ones`, `rand`
  - e.g. `zeros(2,3)` creates a matrix of all zeros with 2 rows and 3 columns:

$$\begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix}$$

- `[1 2 3]` creates the row vector: $\begin{bmatrix} 1 & 2 & 3 \end{bmatrix}$
- `[1; 2; 3]` creates the column vector: $\begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix}$
- `[1 2 3; 4 5 6]` creates the matrix: $\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix}$

Creating more matrices

- `[1 2 3; 4 5]` doesn't work and throws an error. *Every row needs to be the same length.*

Creating more matrices

- `[1 2 3; 4 5]` doesn't work and throws an error. *Every row needs to be the same length.*
- Can do things like `[1 2 3; ones(1,3)]` to get: $\begin{bmatrix} 1 & 2 & 3 \\ 1 & 1 & 1 \end{bmatrix}$

Creating more matrices

- `[1 2 3; 4 5]` doesn't work and throws an error. *Every row needs to be the same length.*
- Can do things like `[1 2 3; ones(1,3)]` to get: $\begin{bmatrix} 1 & 2 & 3 \\ 1 & 1 & 1 \end{bmatrix}$
- More generally, as long as `x` and `y` have the same number of rows, can make a matrix with "y to the right of x" with `[x y]`.

Creating more matrices

- `[1 2 3; 4 5]` doesn't work and throws an error. *Every row needs to be the same length.*
- Can do things like `[1 2 3; ones(1,3)]` to get: $\begin{bmatrix} 1 & 2 & 3 \\ 1 & 1 & 1 \end{bmatrix}$
- More generally, as long as `x` and `y` have the same number of rows, can make a matrix with "y to the right of x" with `[x y]`.
- If `x` and `y` instead have the same number of columns, can put "y below x" with `[x; y]`.

Working with matrices

We're going to define and then work with the matrix

$$M = \begin{bmatrix} 5 & -2 & 3.1 \\ 4 & 9.33 & 0 \end{bmatrix}$$

```
M = [5 -2 3.1; 4 9.33 0];

[nr, nc] = size(M); % nr is number of rows, nc is number of columns

% Can also get size in a single dimension at a time
nr = size(M, 1);
nc = size(M, 2);

disp(M(2,3)); % will print 0
M(1,2) = 2; % will change the -2 to 2 in matrix
```

Activity: minInMatrix

Write a function that takes in a matrix M and finds the minimum value in the matrix.

Write this function using nested for loops and no built in function other than `size`.

Activity: minInMatrix

Write a function that takes in a matrix M and finds the minimum value in the matrix.

Write this function using nested for loops and no built in function other than `size`.

```
function minValue = minInMatrix(M)
% Finds the minimum value in a nonempty matrix M

minValue = M(1,1);

for row = 1:size(M,1)
    for col = 1:size(M,2)
        if M(row,col) < minValue
            minValue = M(row,col);
        end
    end
end

end
```

What if we had done `for x = M`?

```
M = [1 2 3;
     4 5 6];

for nums = M
    disp(nums)
end
```

What if we had done `for x = M`?

```
M = [1 2 3;
     4 5 6];

for nums = M
    disp(nums)
end
```

When the array is a matrix, a MATLAB for loop processes **one column at a time**.

The values of `nums` are:

$$\begin{bmatrix}1\\4\end{bmatrix}, \quad \begin{bmatrix}2\\5\end{bmatrix}, \quad \begin{bmatrix}3\\6\end{bmatrix}$$

Greyscale images

How might we represent this image?

Greyscale images as matrices

A grayscale image is simply a **2-D matrix**.

Greyscale images as matrices

A grayscale image is simply a **2-D matrix**.

- Each matrix entry represents one **pixel**.
- Rows and columns specify the pixel's position.
- The value specifies the pixel's brightness.

Greyscale images as matrices

A grayscale image is simply a **2-D matrix**.

- Each matrix entry represents one **pixel**.
- Rows and columns specify the pixel's position.
- The value specifies the pixel's brightness.

For an image stored using integers from 0 to 255:

$$\mathbf{0} = \text{black}, \qquad \mathbf{255} = \text{white}$$

Values between 0 and 255 produce different shades of gray.

Greyscale images as matrices

A grayscale image is simply a **2-D matrix**.

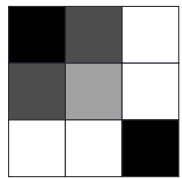
- Each matrix entry represents one **pixel**.
- Rows and columns specify the pixel's position.
- The value specifies the pixel's brightness.

For an image stored using integers from 0 to 255:

$$\mathbf{0} = \text{black}, \qquad \mathbf{255} = \text{white}$$

Values between 0 and 255 produce different shades of gray.

$$M = \begin{bmatrix} 0 & 80 & 255 \\ 80 & 160 & 255 \\ 255 & 255 & 0 \end{bmatrix}$$



`M(row,col)` gives the brightness of one pixel.

Working with greyscale images in MATLAB

```
flower = imread("flower.jpg"); % flower is now a 2d array
size(flower); % will show the dimension of the img in pixels
imshow(flower); % will show us the img using the array
```

Working with greyscale images in MATLAB

```
flower = imread("flower.jpg"); % flower is now a 2d array
size(flower); % will show the dimension of the img in pixels
imshow(flower); % will show us the img using the array
```

We can "crop" the image by indexing with the colon operator.

```
cropped_flower = flower(400:600, 500:1000);
imshow(cropped_flower);
```

`flower(400:600, 500:1000)` means we are getting the submatrix made of:

- the rows 400 through 600,
- the columns 500 through 1000.

Working with greyscale images in MATLAB

```
flower = imread("flower.jpg"); % flower is now a 2d array
size(flower); % will show the dimension of the img in pixels
imshow(flower); % will show us the img using the array
```

We can "crop" the image by indexing with the colon operator.

```
cropped_flower = flower(400:600, 500:1000);
imshow(cropped_flower);
```

`flower(400:600, 500:1000)` means we are getting the submatrix made of:

- the rows 400 through 600,
- the columns 500 through 1000.

`flower(:, 500:1000)` would mean all rows and columns 500 through 1000.

Working with greyscale images in MATLAB

```
1 cropped_flower = flower(400:600, 500:1000);
2 imshow(cropped_flower);
```

Challenge: Mirror the Flower

Activity: Write MATLAB code that creates a left-to-right mirror image of the flower by reflecting it across the image's vertical centerline.

Challenge: Mirror the Flower

Activity: Write MATLAB code that creates a left-to-right mirror image of the flower by reflecting it across the image's vertical centerline.

Solution:

```
imshow(flower(:, 1920:-1:1));
```

Challenge: Mirror the Flower

Activity: Write MATLAB code that creates a left-to-right mirror image of the flower by reflecting it across the image's vertical centerline.

Solution:

```
imshow(flower(:, 1920:-1:1));
```

`end` keyword refers to the final row/column/etc, so could also use:

```
imshow(flower(:, end:-1:1));
```

Inverting colors

What if we want to invert the colors in the image?

Inverting colors means replacing white pixels with black pixels, light gray pixels with dark gray pixels, and so on.

Since brightness is represented by a number between `0` and `255`, we replace each pixel's brightness `x` with `255 - x`.

Inverting colors

What if we want to invert the colors in the image?

Inverting colors means replacing white pixels with black pixels, light gray pixels with dark gray pixels, and so on.

Since brightness is represented by a number between `0` and `255`, we replace each pixel's brightness `x` with `255 - x`.

```
inverted_flower = flower;

for row = 1:size(flower, 1)
    for col = 1:size(flower, 2)
        inverted(row, col) = 255 - flower(row, col);
    end
end

imshow(inverted);
```

Inverting colors: Vectorized operations

```
inverted_flower = flower;

for row = 1:size(flower, 1)
    for col = 1:size(flower, 2)
        inverted(row, col) = 255 - flower(row, col);
    end
end

imshow(inverted);
```

Can we do this more efficiently?

Inverting colors: Vectorized operations

```
inverted_flower = flower;

for row = 1:size(flower, 1)
    for col = 1:size(flower, 2)
        inverted(row, col) = 255 - flower(row, col);
    end
end

imshow(inverted);
```

Can we do this more efficiently?

Just like comparison operators, MATLAB let's us apply basic math operations to entire arrays.

These are called *vectorized operations*.

Vectorized operations

```
a = [1 2; 3 4];
b = [1 0; 0 1];

a + 4; % [5 6; 7 8]
a - 4; % [-3 -2; -1 0]

a + b; % [2 2; 3 5]
```

Vectorized operations

```
a = [1 2; 3 4];
b = [1 0; 0 1];

a + 4; % [5 6; 7 8]
a - 4; % [-3 -2; -1 0]

a + b; % [2 2; 3 5]
```

What will we get if we do `a * b`?

Vectorized operations

```
a = [1 2; 3 4];
b = [1 0; 0 1];

a + 4; % [5 6; 7 8]
a - 4; % [-3 -2; -1 0]

a + b; % [2 2; 3 5]
```

What will we get if we do `a * b`?

$$\begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$$

By default, `*`, `/`, and `^` will be treated as matrix operations, not element-by-element operations.

Vectorized operations

If you want the element-by-element operations, you add a dot for the following operations:

- multiplication is `.*`
- division is `./`
- exponentiation is `.^`

Vectorized operations

If you want the element-by-element operations, you add a dot for the following operations:

- multiplication is `.*`
- division is `./`
- exponentiation is `.^`

```
a = [1 2; 3 4];
b = [1 0; 0 1];

a .* b; % [1 0; 0 4]
a .^ 2; % [1 4; 9 16]
```

What is `a .^ b`?

Vectorized operations

If you want the element-by-element operations, you add a dot for the following operations:

- multiplication is `.*`
- division is `./`
- exponentiation is `.^`

```
a = [1 2; 3 4];
b = [1 0; 0 1];

a .* b; % [1 0; 0 4]
a .^ 2; % [1 4; 9 16]
```

What is `a .^ b`?

`[1 1; 1 4]`

Back to inverting colors

What is the most efficient way to invert the colors for the flower image?

Back to inverting colors

What is the most efficient way to invert the colors for the flower image?

```
inverted_flower = 255 - flower;
imshow(inverted_flower);
```